



Estrutura de Dados não Lineares

Prof. Thiago Koman Schmidt



Planejamento da aula

- Percursos
 - In-Order
 - Pre-Order
 - Post-Order
- Operações em árvores
 - Inserção
 - Remoção



Percursos

Percurso

Um percurso é a forma de visitar sistematicamente todos os nós de uma árvore, garantindo que nenhum nó seja ignorado.

Define a ordem em que os nós são processados (exibidos, salvos, manipulados).

Importância:

- Permite listar os elementos da árvore de diferentes formas
- É base para operações como busca, cópia, exclusão, cálculo de expressões
- Fundamenta estruturas e algoritmos mais complexos (ex.: árvores balanceadas, árvores de expressão, parsing em compiladores)

Percurso

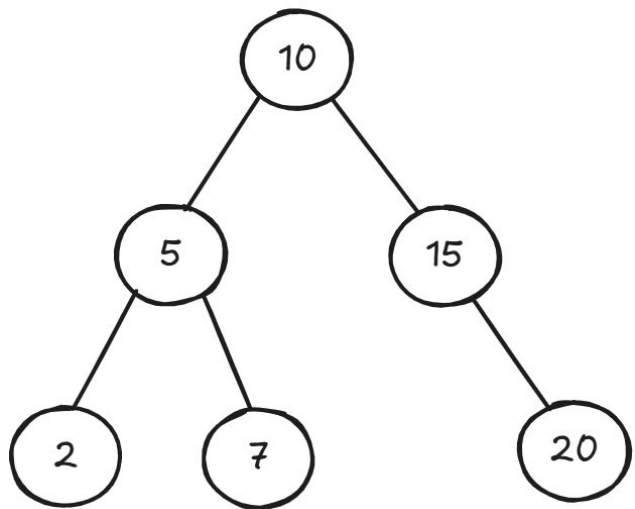
Tipos principais:

- **In-order (Esquerda $\rightarrow$ Raiz $\rightarrow$ Direita):**
 - resulta em ordem crescente em BST.
- **Pre-order (Raiz $\rightarrow$ Esquerda $\rightarrow$ Direita):**
 - usado para gerar/codificar a árvore.
- **Post-order (Esquerda $\rightarrow$ Direita $\rightarrow$ Raiz):**
 - usado em exclusões e avaliação de expressões.

Percurso In-order

Visita: Esquerda → Raiz → Direita

Em Árvores de Busca Binária → retorna os valores ordenados



2, 5, 7, 10, 15, 20 (ordenado).

Percorso In-order

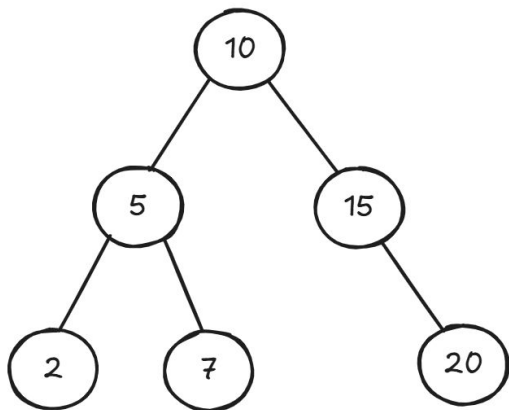
```
void inOrder(Node* root) {  
    if (root) {  
        inOrder(root->left);  
        cout << root->key << " ";  
        inOrder(root->right);  
    }  
}
```

Percurso Pre-order

Visita: Raiz → Esquerda → Direita

Usado em:

- Copiar árvore
- Expressões prefixa



1. Visita a raiz → 10
2. Vai para a esquerda → 5
3. Esquerda de 5 → 2
4. Direita de 5 → 7
5. Vai para a direita da raiz → 15
6. Direita de 15 → 20

10, 5, 2, 7, 15, 20

Percorso Pre-order

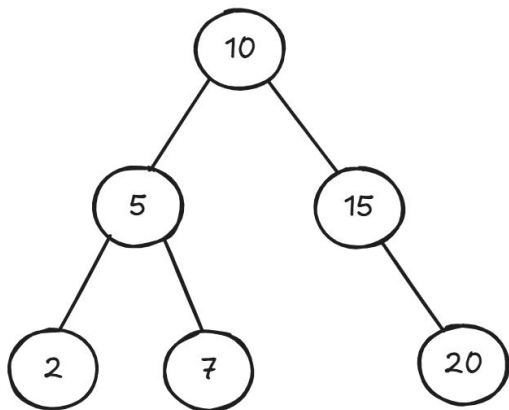
```
void preOrder(Node* root) {  
    if (root) {  
        cout << root->key << " ";  
        preOrder(root->left);  
        preOrder(root->right);  
    }  
}
```

Percurso Post-order

Visita: Esquerda $\rightarrow$ Direita $\rightarrow$ Raiz

Usado em:

- Remover nós (deleta filhos antes do pai)
- Avaliação de expressões



1. Esquerda de 10 $\rightarrow$ 5
2. Esquerda de 5 $\rightarrow$ 2
3. Direita de 5 $\rightarrow$ 7
4. Depois visita o 5
5. Direita de 10 $\rightarrow$ 15
6. Direita de 15 $\rightarrow$ 20
7. Depois visita o 15
8. Por último, a raiz $\rightarrow$ 10

2, 7, 5, 20, 15, 10

Percorso Post-order

```
void postOrder(Node* root) {  
    if (root) {  
        postOrder(root->left);  
        postOrder(root->right);  
        cout << root->key << " ";  
    }  
}
```

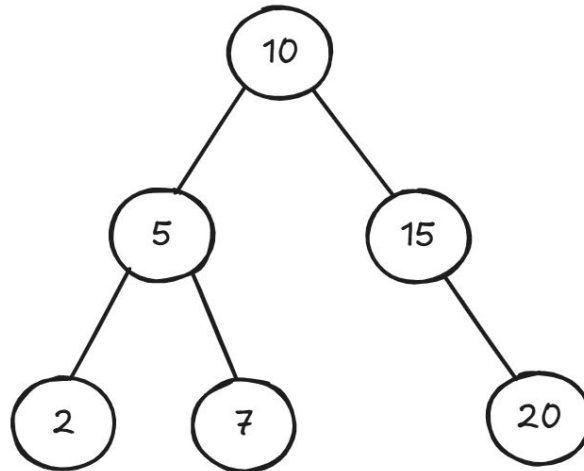


Atividade 1

Atividade 1

Implemente a árvore abaixo e compare a saída dos 3 algoritmos de percurso, in-order, pre-order e post-order.

Faça o envio do código com extensão **cpp**.





Operações em árvores

Operação de Inserção em BST

- Comparar valor com nó atual
- Se menor, vai para esquerda
- Se maior, vai para direita
- Continua até encontrar posição nula

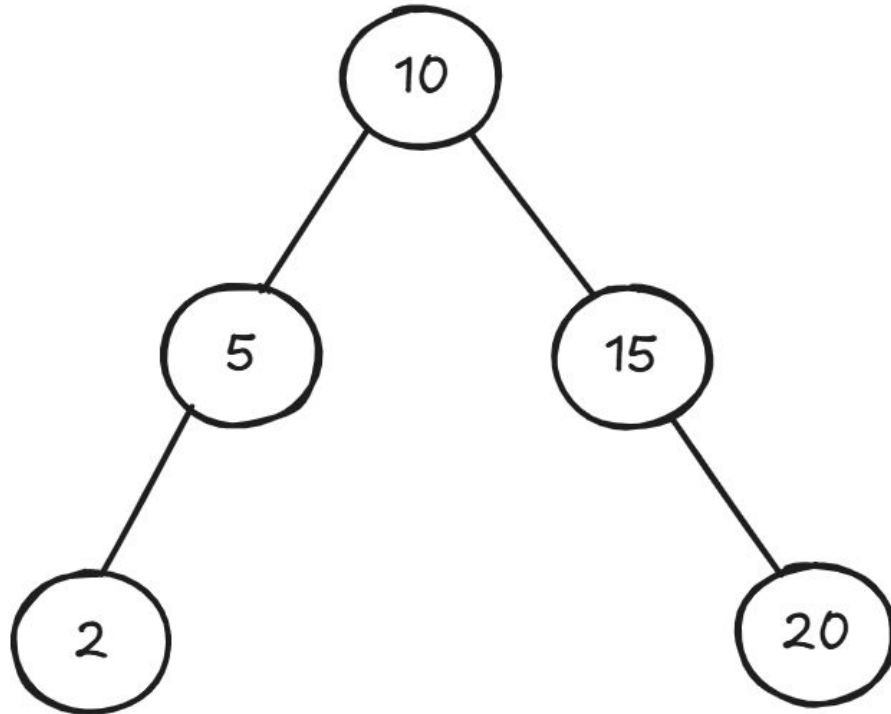
Exemplo:

Inserção de 7.

Operação de Inserção em BST

Exemplo:

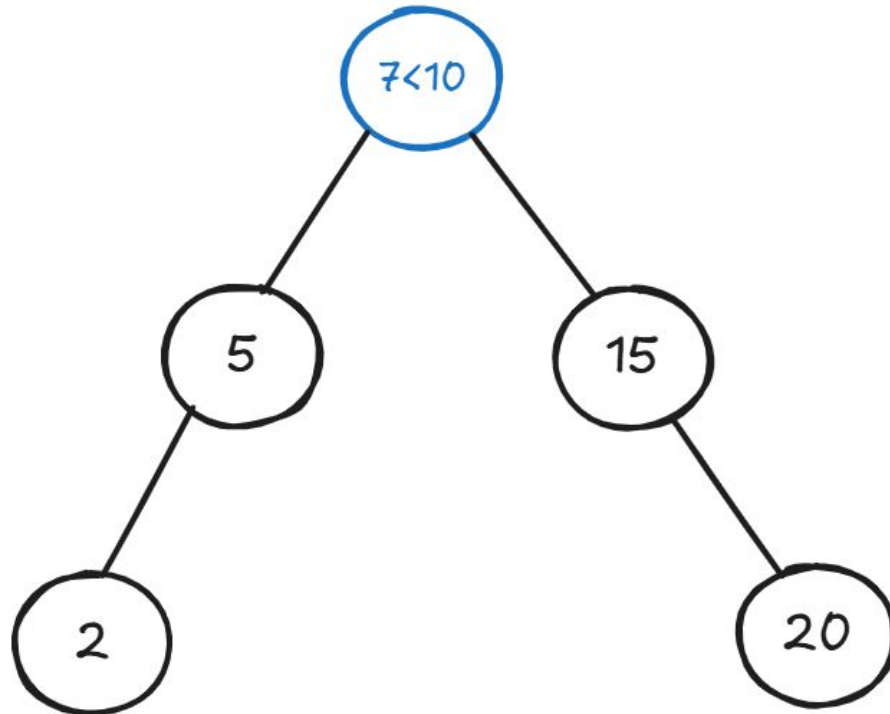
Inserção de 7.



Operação de Inserção em BST

Exemplo:

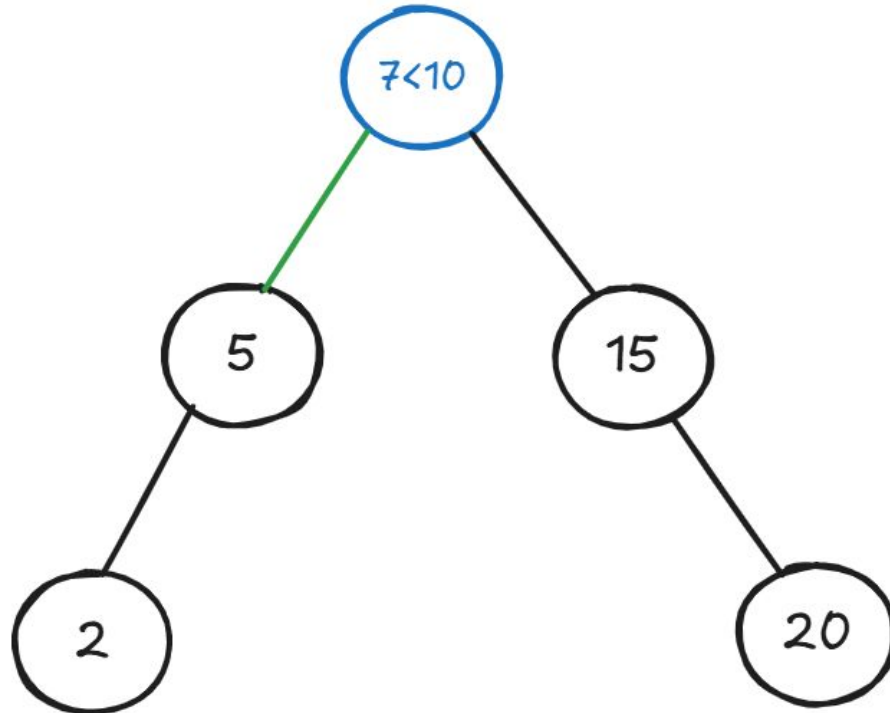
Inserção de 7.



Operação de Inserção em BST

Exemplo:

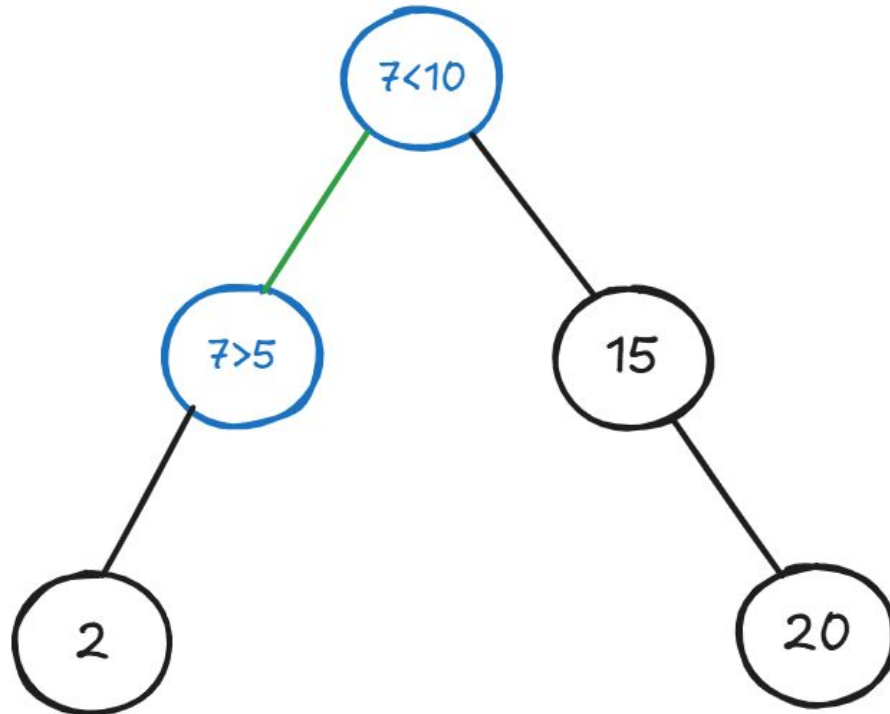
Inserção de 7.



Operação de Inserção em BST

Exemplo:

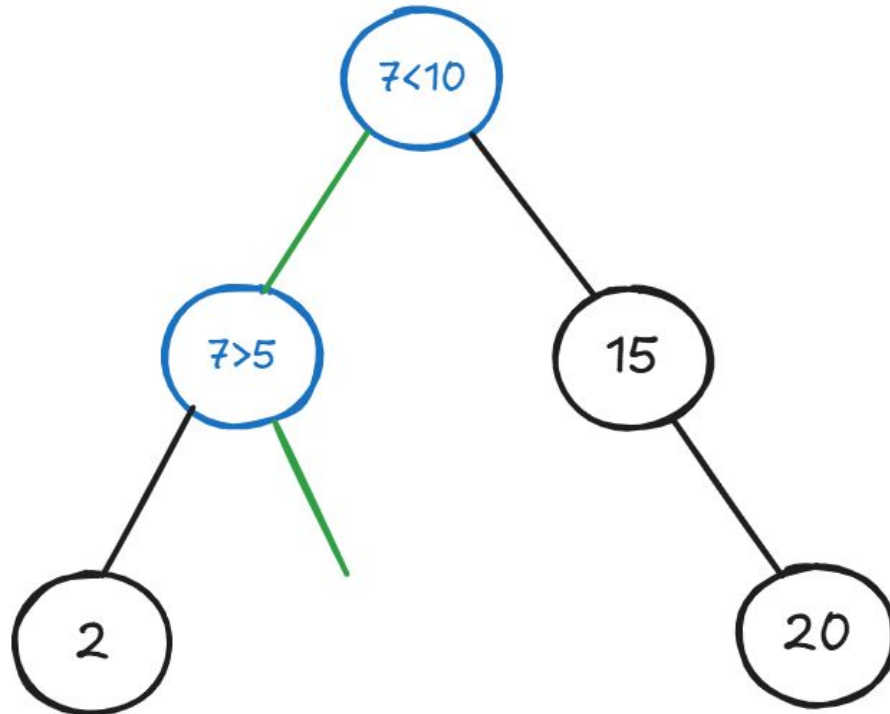
Inserção de 7.



Operação de Inserção em BST

Exemplo:

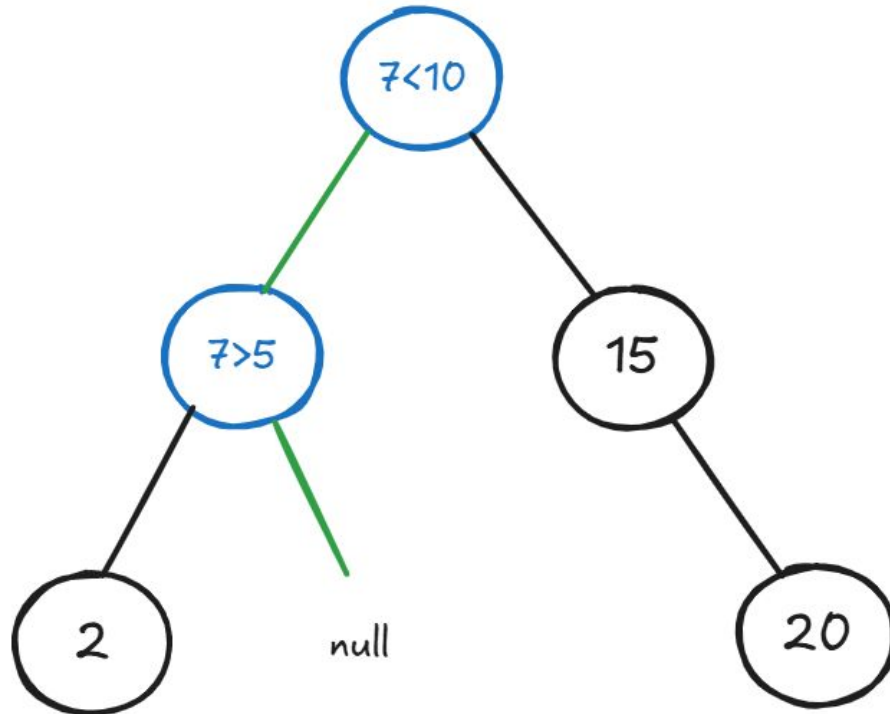
Inserção de 7.



Operação de Inserção em BST

Exemplo:

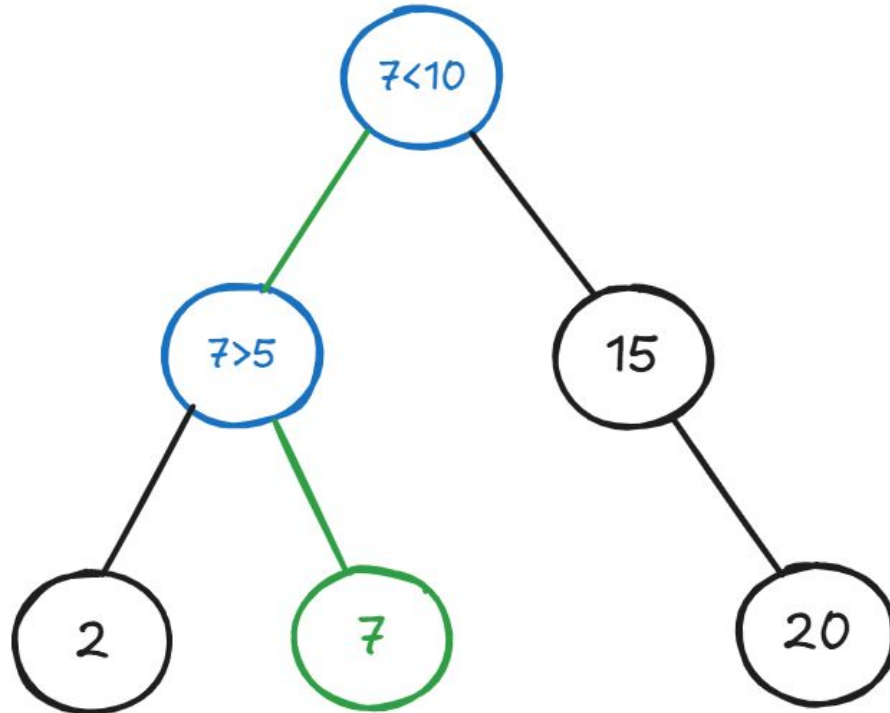
Inserção de 7.



Operação de Inserção em BST

Exemplo:

Inserção de 7.



Operação de Inserção em BST

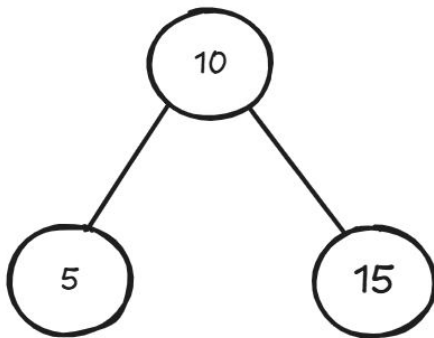
```
Node* insert(Node* root, int key) {  
    if (!root) return new Node(key);  
    if (key < root->key)  
        root->left = insert(root->left, key);  
    else if (key > root->key)  
        root->right = insert(root->right, key);  
    return root;  
}
```

Operação de Remoção em BST

- **Nó folha** → remove direto
- **Nó com 1 filho** → conecta filho ao pai
- **Nó com 2 filhos** → substitui pelo sucessor (menor da direita) ou antecessor (maior da esquerda)

Exemplo:

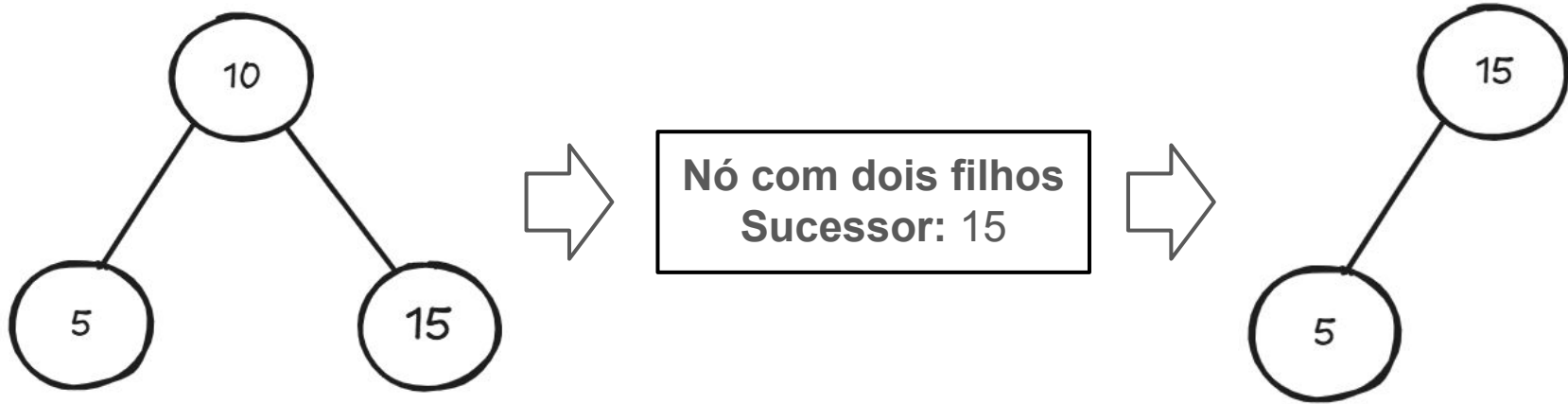
Remover 10 da árvore



Operação de Remoção em BST

Exemplo:

Remover 10 da árvore



Função de remoção

```
Node* remove(Node* root, int key) {
    if (!root) return root;

    if (key < root->key){
        root->left = remove(root->left, key);
    }else if (key > root->key){
        root->right = remove(root->right, key);
    }else {
        // Caso 1 e 2: nó com 0 ou 1 filho
        if (!root->left){ return root->right; }
        if (!root->right){ return root->left; }

        // Caso 3: dois filhos
        Node* succ = root->right;
        while (succ->left){
            succ = succ->left;
        }
        root->key = succ->key;
        root->right = remove(root->right, succ->key);
    }
    return root;
}
```

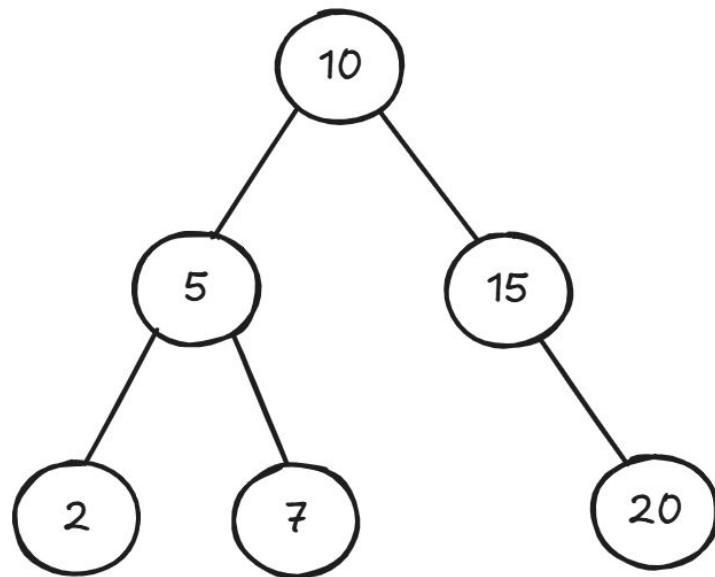


Atividade 2

Atividade 2

Implemente a árvore abaixo e utilize a função de inserção para incluir o valor **12** na árvore binária de busca. Agora utilize o algoritmo de exclusão para remover o nó com valor **15**.

Faça o envio do código com extensão **cpp**.





Conclusão

Aplicações Práticas de Árvores

Compiladores → árvore de sintaxe abstrata

Bancos de dados → índices com árvores B / B+

Sistemas de arquivos → diretórios em árvore

Algoritmos de busca → árvores de decisão

Expressões matemáticas → árvores de expressão

Classificação de Dados com IA → árvores de decisão

DÚVIDAS?

